Decentralized Multi-Agent Path Finding under Dynamic Communication Constraints

Project Proposal : Multi-Agent Systems

Ronit Ranjan (AI24BTECH11028) · Harshvardhan Patidar (AI24BTECH11015) · Charitha Sri (AI24BTECH11014)
Kallem Maanya Sri (AI24BTECH11017) · Yadlapally Shreedhanvi (AI24BTECH11036)
Indian Institute of Technology Hyderabad

1. Problem and Research Question

Consider N mobile robots on a shared grid, each with its own start and goal, which must all arrive without obstructing one another. Solved *centrally*, by a single planner with full knowledge of every robot, this is classical Multi-Agent Path Finding (MAPF), and existing solvers already scale to thousands of agents. Real deployments do not work this way. A warehouse fleet or drone swarm has no omniscient controller: each robot senses only its surroundings and exchanges messages only with robots nearby, over a radio link of finite range and bandwidth that drops packets. Coordination must emerge from local interaction over a *communication graph* that changes shape whenever the robots move.

Research question. *How much communication does decentralized multi-robot coordination actually require, and how gracefully does coordination degrade as that communication becomes sparse, delayed, or unreliable?*

2. Platform

We found **POGEMA** [3] (Github) a useful existing repository to build on: a fast grid-world simulator in which many agents navigate to individual goals under a partially-observable field of view, with seeded reproducible instances and built-in success-rate, makespan and sum-of-costs metrics. What it does not provide is any notion of communication: no message, no communication graph, no channel model and no failure model. That layer is precisely our contribution.

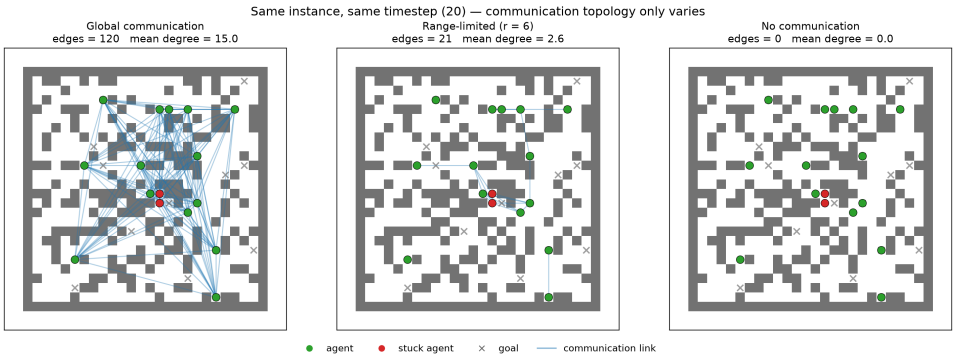


Figure 1: Working output from our extended POGEMA setup. Same instance and timestep in all three panels; only the communication graph differs, with mean degree falling 15.0 / 2.6 / 0. Red markers are deadlocked agents.

3. Committed Core

The nine milestones below are what we undertake to deliver. They are ordered, i.e. each depends only on those before it, and together they constitute a complete project on their own.

1. **Environment setup.** POGEMA installed and verified at 100 agents, with two facts established that shape everything after: observation and communication are separate variables, and collisions are structurally impossible, so *deadlock* is the failure mode we measure.
2. **Communication layer.** A function from agent positions to a directed adjacency structure, specifying who can transmit to whom at each timestep, with the two bounding topologies: *global* (control condition) and *none* (performance floor).
3. **Channel interface.** A no-op stage through which every message passes, built early so that the failure modes added later require no refactor.
4. **Decentralized planner.** Per-robot A* plus a local yield rule on conflicts with known neighbours, following the published DCMAPF design [1,2]. One planner serves all topologies: it reads its inbox and does not know why the inbox is full or empty, so any performance change is attributable to *information* alone.
5. **Deadlock detection and metrics.** Success rate (individual and cooperative), makespan, sum-of-costs, deadlock count and message count.
6. **Oracle and floor.** A centralized near-optimal ceiling (LaCAM3 [4]) and the zero-communication floor, between which every decentralized result is reported as a ratio.
7. **Experiment harness.** Configuration-driven sweeps with every run seeded, settings compared on *matched instances* (identical map, starts and goals), raw results persisted before aggregation, and error bars over ≥ 5 seeds.
8. **Range-limited and k-NN topologies.** The remaining two of our four topologies, compared at matched mean degree, with degree, component count and connectivity logged alongside every performance number, since this is what lets us explain results structurally, not merely describe them.

9. **Sweeps, analysis and report.** Topology × communication range × agent count ($N \in \{10, 25, 50, 100\}$) across three map types and five seeds, yielding our headline curves: success rate against communication range, bounded above by the oracle and below by zero communication.

4. Extensions

Attempted only once the core above is finished, in the priority order given. Each is independently droppable, and removing one affects nothing else.

1. **Packet loss.** Each message dropped independently with probability p, swept from 0 to 0.8. The question of interest is the *shape* of the curve: graceful decay, or collapse past a critical rate.
2. **Message delay.** Delivery postponed d timesteps via a queue in the channel stage, so robots plan against stale neighbour positions. This is distinct from the published work on *movement* delays in MAPF: there the robot arrives late, here the message does.
3. **Consensus framing.** Instrumenting the conflict-resolution step as a local consensus protocol and measuring rounds-to-agreement against communication quality.
4. **Behavioural guarantee.** A proof that under full communication and message delay bounded by d, our yield rule cannot deadlock, with the assumption that packet loss violates stated explicitly.

5. Demonstration and Deliverables

Our demonstration has two halves. **Qualitatively**, Figure 1 animated across a full episode: the communication graph forming and fragmenting as robots move, side by side across topologies on an identical instance, so the effect of communication is seen directly rather than inferred from a table**Quantitatively**, success rate plotted against communication range and against message-loss probability, in both cases bracketed by the oracle above and zero communication below, with error bars over seeds.

Final deliverables. The extended simulator and experiment code; the quantitative results above across agent counts and map types; the animated topology demonstration; and a written report.

6. Tentative Individual Contributions

Module interfaces are defined and frozen at the outset, so the five workstreams can proceed in parallel against fixed signatures rather than waiting on one another. The oracle and the experiment harness depend on none of the others and begin immediately; the planner needs only the communication and channel signatures; the remaining topologies extend the communication layer. Every member owns both a built component and a reported result.

Member	Responsibility	Result reported
Ronit Ranjan	Environment setup and verification; the communication layer with the global and none topologies; the channel interface; the simulation loop and frozen module interfaces	Three-bar validation: oracle vs. full vs. no communication (with Harshvardhan)
Harshvardhan Patidar	The decentralized planner and its local yield rule; deadlock detection; the metric suite	Three-bar validation (with Ronit); deadlock characterisation : where and how often coordination fails
Yadlapally Shreedhanvi	The centralized oracle (LaCAM3 build) and the zero-communication floor	Cost-of-decentralization ratios; **overall synthesis** of the findings into a single answer to the research question
Charitha Sri	The experiment harness: seeding, matched-instance discipline, sweep execution and the plotting pipeline	Agent-count scaling: how the communication requirement grows with fleet density
Kallem Maanya Sri	Range-limited and k-nearest-neighbour topologies at matched mean degree; graph-statistics instrumentation	**Headline curves**: success rate against communication range, and the topology comparison

References

1. A. Maoudj, A. L. Christensen. Decentralized MAPF in Warehouse Environments for Fleets of Mobile Robots with Limited Communication Range. *ANTS 2022*.
2. A. Maoudj, A. L. Christensen. Improved decentralized cooperative MAPF for robots with limited communication. *Swarm Intelligence*, 2023.
3. A. Skrynnik et al. POGEMA: A Benchmark Platform for Cooperative Multi-Agent Pathfinding. *ICLR 2025*. arXiv:2407.14931.
4. K. Okumura. Engineering LaCAM*: Towards Real-Time, Large-Scale, Near-Optimal MAPF. *AAMAS 2024*.